

Applied Data Science Project 1: Motor Vehicle Crashes in NYC

Maggie Boles, Adina Kaplan, Logan McCorkle, & Chana Ochs

03/28/2026

The purpose of this code is to clean and process datasets from the following sources to be used later in our model

Source 1 (NYC Motor Vehicle Crashes): <https://catalog.data.gov/dataset/motor-vehicle-collisions-crashes>
Source 2 (Traffic Signal and Stop Requests): <https://data.cityofnewyork.us/Transportation/Traffic-Signal-and-All-Way-Stop-Study-F>
Source 3 (Traffic Signal Condition): https://data.cityofnewyork.us/Social-Services/Traffic-Signal-Condition_4/e5mv-wy8f/about_dat
Source 4 (NYS DOT Maintained Traffic Signals): https://data.gis.ny.gov/datasets/c61a41873d4f4b06bae79c23ad98b9c6_0/about

```
#NYC Motor Vehicle Crashes
# Import Libraries
import pandas as pd
from datetime import datetime

# Load CSV
df = pd.read_csv('Motor_Vehicle_Collisions.csv')

# 1. Standardize CRASH DATE → datetime.date (or full datetime)
df['CRASH DATE'] = pd.to_datetime(df['CRASH DATE'], errors='coerce').dt.date

# 2. Standardize CRASH TIME → datetime.time (or full datetime)
df['CRASH TIME'] = pd.to_datetime(df['CRASH TIME'], format='%H:%M', errors='coerce').dt.time

# 3. Create a full datetime column (very useful)
df['CRASH_DATETIME'] = pd.to_datetime(
    df['CRASH DATE'].astype(str) + ' ' + df['CRASH TIME'].astype(str),
    errors='coerce'
)

# 4. Extract Day of the Week
df['DAY_OF_WEEK'] = df['CRASH_DATETIME'].dt.day_name()      # e.g., "Monday", "Tuesday"
# OR numerical version (0 = Monday, 6 = Sunday)
df['DAY_OF_WEEK_NUM'] = df['CRASH_DATETIME'].dt.weekday

# 5. Add abbreviated day name
df['DAY_OF_WEEK_ABBR'] = df['CRASH_DATETIME'].dt.day_name().str[:3]

# Check the results
print(df[['CRASH DATE', 'CRASH TIME', 'CRASH_DATETIME', 'DAY_OF_WEEK']].head())

# Save the cleaned version
df.to_csv('nyc_crashes_cleaned.csv', index=False)
```

```
print("\n✅ File saved as: nyc_crashes_cleaned.csv")
```

	CRASH DATE	CRASH TIME	CRASH_DATETIME	DAY_OF_WEEK
0	2021-09-11	02:39:00	2021-09-11 02:39:00	Saturday
1	2022-03-26	11:45:00	2022-03-26 11:45:00	Saturday
2	2023-11-01	01:29:00	2023-11-01 01:29:00	Wednesday
3	2022-06-29	06:55:00	2022-06-29 06:55:00	Wednesday
4	2022-09-21	13:21:00	2022-09-21 13:21:00	Wednesday

```
✅ File saved as: nyc_crashes_cleaned.csv
```

```
#NYC Motor Vehicle Crashes
```

```
# 6. Drop rows with missing Latitude or Longitude
```

```
df = df.dropna(subset=['LATITUDE', 'LONGITUDE']).reset_index(drop=True)
```

```
# 7. Filter data from 2022 to 03/28/2026
```

```
start_date = pd.to_datetime('2022-01-01').date()
```

```
end_date = pd.to_datetime('2026-03-28').date()
```

```
df = df[
    (df['CRASH DATE'] >= start_date) &
    (df['CRASH DATE'] <= end_date)
].reset_index(drop=True)
```

```
# Add Year and Month for easier analysis
```

```
df['YEAR'] = df['CRASH_DATETIME'].dt.year
```

```
df['MONTH'] = df['CRASH_DATETIME'].dt.month_name()
```

```
# Check the results
```

```
print(f"Original rows: {len(pd.read_csv('nyc_crashes_cleaned.csv'))},}") # for reference
```

```
print(f"Rows after cleaning and filtering: {len(df)},}")
```

```
print(f>Date range: {df['CRASH DATE'].min()} to {df['CRASH DATE'].max()}")
```

```
print("\nFirst 5 rows of key columns:")
```

```
print(df[['CRASH DATE', 'CRASH TIME', 'DAY_OF_WEEK', 'LATITUDE', 'LONGITUDE', 'YEAR']].head())
```

```
# Save the cleaned and filtered dataset
```

```
df.to_csv('nyc_crashes_cleaned_2022_to_2026.csv', index=False)
```

```
print("\n✅ File saved as: nyc_crashes_cleaned_2022_to_2026.csv")
```

```
Original rows: 655,771
```

```
Rows after cleaning and filtering: 229,626
```

```
Date range: 2022-01-01 to 2025-06-05
```

```
First 5 rows of key columns:
```

	CRASH DATE	CRASH TIME	DAY_OF_WEEK	LATITUDE	LONGITUDE	YEAR
0	2023-11-01	01:29:00	Wednesday	40.621790	-73.970024	2023
1	2022-09-22	16:16:00	Thursday	40.698257	-73.826320	2022
2	2022-07-12	17:50:00	Tuesday	40.663303	-73.960490	2022
3	2022-04-24	16:45:00	Sunday	40.607685	-74.138920	2022
4	2022-04-24	04:49:00	Sunday	40.855972	-73.869896	2022

```
✅ File saved as: nyc_crashes_cleaned_2022_to_2026.csv
```

```
#Traffic Signal and Stop Requests
```

```
# Import Libraries
```

```
import pandas as pd
```

```

# Load the dataset
df_requests = pd.read_csv('Traffic_Signal_Requests.csv')

print(f"Original shape: {df_requests.shape}")

# 1. Standardize Date Columns
date_columns = ['DateCreated', 'DateRequested', 'StatusDate', 'DateAssigned',
                'TargetDate', 'ReEvaluationDate', 'SignalInstallDate',
                'AW_InstallDate', 'TentativeSignalInstallDate', 'ATR_ReceiveDate',
                'ATR_RequestDate', 'WarrantsSatisfiedDate']

for col in date_columns:
    if col in df_requests.columns:
        df_requests[col] = pd.to_datetime(df_requests[col], errors='coerce')

# 2. Extract useful time features
df_requests['REQUEST_YEAR'] = df_requests['DateRequested'].dt.year
df_requests['REQUEST_MONTH'] = df_requests['DateRequested'].dt.month_name()
df_requests['DAY_OF_WEEK'] = df_requests['DateRequested'].dt.day_name()

# 3. Clean Text Columns
text_cols = ['Borough', 'StatusDescription', 'StudyStatus', 'RequestType',
             'LocationType', 'MainStreet', 'CrossStreet1', 'CrossStreet2']

for col in text_cols:
    if col in df_requests.columns:
        df_requests[col] = df_requests[col].astype(str).str.strip().str.upper()

# 4. Create a clean combined location field (very useful)
df_requests['FULL_LOCATION'] = (
    df_requests['MainStreet'].fillna('') + " @ " +
    df_requests['CrossStreet1'].fillna('') +
    df_requests['CrossStreet2'].apply(lambda x: f" & {x}" if pd.notna(x) and x != '' else '')
).str.strip()

# 5. Handle Boolean / Yes-No columns consistently
yes_no_cols = ['VisionZero', 'Seasonal', 'HurricaneSandy', 'SchoolHold',
               'SchoolCrossing', 'Removal', 'MidBlock']

for col in yes_no_cols:
    if col in df_requests.columns:
        df_requests[col] = df_requests[col].astype(str).str.upper().map({
            'YES': True, 'Y': True, 'TRUE': True,
            'NO': False, 'N': False, 'FALSE': False
        }).fillna(False)

# 6. Filter to more recent / relevant requests (optional but recommended)
# Keep only requests from 2015 onward (adjust as needed)
df_requests = df_requests[df_requests['DateRequested'].dt.year >= 2015].reset_index(drop=True)

# Final summary
print(f"Cleaned shape: {df_requests.shape}")
print(f>Date range (DateRequested): {df_requests['DateRequested'].min()} to {df_requests['DateRequested'].max()}")
print(f>Boroughs: {df_requests['Borough'].unique()}")

print("\nSample of key columns:")

```

```
print(df_requests[['Id', 'DateRequested', 'Borough', 'RequestType', 'StatusDescription',
                  'FULL_LOCATION', 'REQUEST_YEAR', 'DAY_OF_WEEK']].head())
```

```
# Save cleaned version
df_requests.to_csv('Traffic_Signal_Requests_cleaned.csv', index=False)
print("\n✅ File saved as: Traffic_Signal_Requests_cleaned.csv")
```

```
Original shape: (74715, 57)
Cleaned shape: (35533, 61)
Date range (DateRequested): 2015-01-02 00:00:00 to 2026-02-06 00:00:00
Boroughs: <StringArray>
[    'QUEENS',      'BRONX',      'BROOKLYN',      'MANHATTAN',
  'STATEN ISLAND',  'VARIOUS',  'ALL BOROUGHs',      'N',
    'BQ',          'MQ',          'QM',          nan]
Length: 12, dtype: str
```

Sample of key columns:

	Id	DateRequested	Borough	RequestType
0	228428	2025-07-28	QUEENS	TRAFFIC SIGNAL
1	219220	2025-04-29	BRONX	TRAFFIC SIGNAL
2	133369	2016-08-03	QUEENS	TRAFFIC SIGNAL
3	230070	2025-10-16	BROOKLYN	TRAFFIC SIGNAL
4	228624	2025-08-29	MANHATTAN	TRAFFIC SIGNAL

	StatusDescription
0	ENGINEERING STUDY COMPLETED (SIGNALS ENGIN...
1	STUDY REQUEST DENIAL
2	ENGINEERING STUDY COMPLETED (SIGNALS ENGIN...
3	ENGINEERING STUDY COMPLETED (SIGNALS ENGIN...
4	ENGINEERING STUDY COMPLETED (SIGNALS ENGIN...

	FULL_LOCATION	REQUEST_YEAR	DAY_OF_WEEK
0	GUY R BREWER BOULEVARD @ ROCKAWAY BOULEVARD & ...	2025	Monday
1	WEST 250 STREET @ INDEPENDENCE AVENUE	2025	Tuesday
2	36 AVENUE @ 37 STREET	2016	Wednesday
3	COURT STREET @ HAMILTON AVENUE & WEST 9 STREET	2025	Thursday
4	SEAMAN AVENUE @ DYCKMAN STREET	2025	Friday

✅ File saved as: Traffic_Signal_Requests_cleaned.csv

Start coding or [generate](#) with AI.

```
#Traffic Signal Condition
# Import Libraries
import pandas as pd

# Load the file
df_condition = pd.read_csv('Traffic_Signal_Condition.csv', low_memory=False)

print(f"Original shape: {df_condition.shape}")

# 1. Standardize Column Names (make them clean and consistent)
df_condition = df_condition.rename(columns={
    'Unique Key': 'UNIQUE_KEY',
    'Created Date': 'CREATED_DATE',
    'Closed Date': 'CLOSED_DATE',
    'Problem (formerly Complaint Type)': 'PROBLEM',
```

```

'Problem Detail (formerly Descriptor)': 'PROBLEM_DETAIL',
'Location Type': 'LOCATION_TYPE',
'Incident Zip': 'INCIDENT_ZIP',
'Incident Address': 'INCIDENT_ADDRESS',
'Street Name': 'STREET_NAME',
'Cross Street 1': 'CROSS_STREET_1',
'Cross Street 2': 'CROSS_STREET_2',
'Intersection Street 1': 'INTERSECTION_STREET_1',
'Intersection Street 2': 'INTERSECTION_STREET_2',
'X Coordinate (State Plane)': 'X_STATEPLANE',
'Y Coordinate (State Plane)': 'Y_STATEPLANE',
'Latitude': 'LATITUDE',
'Longitude': 'LONGITUDE',
'Borough': 'BOROUGH',
'City': 'CITY',
'Status': 'STATUS',
'Due Date': 'DUE_DATE',
'Updated Date': 'UPDATED_DATE',
'Resolution Action': 'RESOLUTION_ACTION',
'Community Board': 'COMMUNITY_BOARD'
})

# 2. Convert Date Columns to proper datetime
date_cols = ['CREATED_DATE', 'CLOSED_DATE', 'DUE_DATE', 'UPDATED_DATE']

for col in date_cols:
    if col in df_condition.columns:
        df_condition[col] = pd.to_datetime(df_condition[col], errors='coerce')

# 3. Create useful derived columns
df_condition['CREATED_YEAR'] = df_condition['CREATED_DATE'].dt.year
df_condition['CREATED_MONTH'] = df_condition['CREATED_DATE'].dt.month_name()
df_condition['CREATED_DAY_OF_WEEK'] = df_condition['CREATED_DATE'].dt.day_name()

# Time to close (in days) - very useful for analysis
if 'CLOSED_DATE' in df_condition.columns and 'CREATED_DATE' in df_condition.columns:
    df_condition['DAYS_TO_CLOSE'] = (df_condition['CLOSED_DATE'] - df_condition['CREATED_DATE']).dt.days

# 4. Clean Coordinate Columns
df_condition['LATITUDE'] = pd.to_numeric(df_condition['LATITUDE'], errors='coerce')
df_condition['LONGITUDE'] = pd.to_numeric(df_condition['LONGITUDE'], errors='coerce')
df_condition['X_STATEPLANE'] = pd.to_numeric(df_condition['X_STATEPLANE'], errors='coerce')
df_condition['Y_STATEPLANE'] = pd.to_numeric(df_condition['Y_STATEPLANE'], errors='coerce')

# Drop rows with missing lat/long (critical for mapping)
df_condition = df_condition.dropna(subset=['LATITUDE', 'LONGITUDE']).reset_index(drop=True)

# 5. Clean Text Columns
text_cols = ['BOROUGH', 'CITY', 'PROBLEM', 'PROBLEM_DETAIL', 'STATUS',
             'LOCATION_TYPE', 'STREET_NAME', 'INCIDENT_ADDRESS']

for col in text_cols:
    if col in df_condition.columns:
        df_condition[col] = df_condition[col].astype(str).str.strip().str.upper()

# Create a clean location string for easier searching

```

```

df_condition['FULL_LOCATION'] = (
    df_condition['STREET_NAME'].fillna('') +
    df_condition['CROSS_STREET_1'].apply(lambda x: f" @ {x}" if pd.notna(x) and x != '' and x != 'nan' else '')
).str.strip()

# 6. Filter to recent years for file size
# Keep only records from 2020 onward (change as needed)
df_condition = df_condition[df_condition['CREATED_DATE'].dt.year >= 2020].reset_index(drop=True)

# Final summary
print(f"Cleaned shape: {df_condition.shape}")
print(f"Date range: {df_condition['CREATED_DATE'].min()} to {df_condition['CREATED_DATE'].max()}")
print(f"Unique Boroughs: {df_condition['BOROUGH'].unique()}")

print("\nSample of cleaned data:")
print(df_condition[['UNIQUE_KEY', 'CREATED_DATE', 'CLOSED_DATE', 'DAYS_TO_CLOSE',
                    'BOROUGH', 'PROBLEM', 'FULL_LOCATION', 'LATITUDE', 'LONGITUDE']].head())

# Save the cleaned version
df_condition.to_csv('Traffic_Signal_Condition_cleaned.csv', index=False)
print("\n✅ File saved as: Traffic_Signal_Condition_cleaned.csv")

```

Original shape: (258995, 38)

C:\Users\maggi\AppData\Local\Temp\ipykernel_32848\1824507813.py:43: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `df\_condition[col] = pd.to\_datetime(df\_condition[col], errors='coerce')`

C:\Users\maggi\AppData\Local\Temp\ipykernel_32848\1824507813.py:43: UserWarning: Could not infer format, so each element will be parsed individually, falling back to `df\_condition[col] = pd.to\_datetime(df\_condition[col], errors='coerce')`

Cleaned shape: (222092, 43)

Date range: 2020-01-01 00:20:00 to 2026-03-27 00:49:00

Unique Boroughs: <StringArray>

['QUEENS', 'MANHATTAN', 'BROOKLYN', 'BRONX', 'STATEN ISLAND', 'UNSPECIFIED']

Length: 6, dtype: str

Sample of cleaned data:

	UNIQUE_KEY	CREATED_DATE	CLOSED_DATE	DAYS_TO_CLOSE	BOROUGH	\
0	68462699	2026-03-27 00:49:00	NaT	NaN	QUEENS	
1	68467220	2026-03-27 00:26:00	NaT	NaN	MANHATTAN	
2	68468752	2026-03-26 23:55:00	NaT	NaN	BROOKLYN	
3	68465713	2026-03-26 23:37:00	NaT	NaN	MANHATTAN	
4	68460691	2026-03-26 23:09:00	NaT	NaN	QUEENS	

	PROBLEM	FULL_LOCATION	LATITUDE	LONGITUDE
0	TRAFFIC SIGNAL CONDITION		40.679015	-73.758671
1	TRAFFIC SIGNAL CONDITION		40.766667	-73.967065
2	TRAFFIC SIGNAL CONDITION		40.605179	-73.980171
3	TRAFFIC SIGNAL CONDITION		40.811442	-73.952102
4	TRAFFIC SIGNAL CONDITION		40.698824	-73.847003

✅ File saved as: Traffic_Signal_Condition_cleaned.csv

#NYS DOT Maintained Traffic Signals

Import Libraries

Load the full file

df_signals = pd.read_csv('NYSDOT_Traffic_Signals.csv')

print(f"Original shape: {df_signals.shape}")

```

# 1. Standardize Coordinate Columns
# Many rows have both (X, Y) and (XCoordinate, YCoordinate). We'll keep the more reliable ones.
df_signals = df_signals.rename(columns={
    'X': 'X_STATEPLANE',
    'Y': 'Y_STATEPLANE',
    'XCoordinate': 'LONGITUDE',
    'YCoordinate': 'LATITUDE'
})

# 2. Convert Latitude & Longitude to proper numeric (some may be strings)
df_signals['LATITUDE'] = pd.to_numeric(df_signals['LATITUDE'], errors='coerce')
df_signals['LONGITUDE'] = pd.to_numeric(df_signals['LONGITUDE'], errors='coerce')

# Drop any rows missing both lat/long (very rare in this dataset, but safe)
df_signals = df_signals.dropna(subset=['LATITUDE', 'LONGITUDE']).reset_index(drop=True)

# 3. Standardize Date Columns
date_cols = ['InstallationDate', 'RebuiltDate', 'RecordCreatedDate',
             'RecordLastModified', 'ServiceLastUpdatedUTC']

for col in date_cols:
    if col in df_signals.columns:
        # Handle formats like "YYYY/MM/DD HH:MM:SS+00"
        df_signals[col] = pd.to_datetime(df_signals[col], errors='coerce')

# 4. Extract useful time features from key dates
if 'InstallationDate' in df_signals.columns:
    df_signals['INSTALL_YEAR'] = df_signals['InstallationDate'].dt.year
    df_signals['INSTALL_MONTH'] = df_signals['InstallationDate'].dt.month_name()

# 5. Clean SignalID and LocationDescription (strip extra spaces)
df_signals['SignalID'] = df_signals['SignalID'].astype(str).str.strip()
df_signals['LocationDescription'] = df_signals['LocationDescription'].astype(str).str.strip()

# 6. Optional: Create a clean "Full Location" column
df_signals['FULL_LOCATION'] = df_signals['LocationDescription'].fillna('') + \
    df_signals['MainRoute'].fillna('')

# Final check
print(f"Cleaned shape: {df_signals.shape}")
print(f"Date range (InstallationDate): {df_signals['InstallationDate'].min()} to {df_signals['InstallationDate'].max()}")
print("\nColumns available now:")
print(df_signals.columns.tolist())

# Save the cleaned version
df_signals.to_csv('NYSDOT_Traffic_Signals_cleaned.csv', index=False)
print("\n🟢 File saved as: NYSDOT_Traffic_Signals_cleaned.csv")

```

```

Original shape: (11277, 65)
Cleaned shape: (11277, 68)
Date range (InstallationDate): 1900-01-01 00:00:00+00:00 to 2027-12-31 00:00:00+00:00

```

```

Columns available now:
['X_STATEPLANE', 'Y_STATEPLANE', 'OBJECTID', 'SignalID', 'LocationDescription', 'DOT_RegionNumber', 'DOT_RegionName', 'SignalShopNumber', 'CountyName', 'CountyFIPS', '
🟢 File saved as: NYSDOT_Traffic_Signals_cleaned.csv

```

Start coding or [generate](#) with AI.